

# Array Methods & Logic

# Array.forEach()

- Array-ийн элемент бүр дээр эхнээс нь авхуулаад ямар нэгэн үйлдэл хийнэ
- Callback function-ийг дуудаж ашиглаж болох ба inline-аар ч бичиж болно
- Parameter-д n буюу ганц утга дээр ажиллахаар заасан бол array-д ямар нэгэн өөрчлөлт орохгүй
- Харин (i,n,arr) гэж зааж өгөөд функцэд ашиглавал array-д өөрчлөлт орно.

```
const food = [ , , , , , , ,  ]
```

```
food.forEach( (item) => { console.log( item ) } );
```



```
array.forEach( cbFunc );
```

```
array.forEach(( element ) => {  
    /* code ... */  
});
```

# Array.map()

- Array-ийн элемент бүр дээр эхнээс нь авхуулаад шинэ array-д буулгадаг.
- Ингэхдээ заавал утга буцаадаг
- Элементүүд дээр үйлдэл хийж болно

```
const food = [ 🍎, 🥕, 🍌, 🍆, 🌽, 🍅, 🥔, 🥑 ]
```

```
const bananas = food.map( (item) => '🍌' );
```

```
bananas → [ 🍌, 🍌, 🍌, 🍌, 🍌, 🍌, 🍌, 🍌 ]
```

```
const duplicateFood = food.map( (any) => any + any );
```

```
duplicateFood → [ 🍎 + 🍎, 🥕 + 🥕, 🍌 + 🍌, 🍆 + 🍆, 🌽 + 🌽, 🍅 + 🍅, 🥔 + 🥔, 🥑 + 🥑 ]
```

```
array.map( ( element ) => {  
  /* code */  
  return /* new element */  
});
```

# Array.filter()

- Array-ийн элемент бүр дээр эхнээс нь авхуулаад шинэ array-д буулгадаг
- Гэхдээ map-аас ялгаатай нь буцах утга дээр ямар нэгэн нөхцөл буцааж array-н элементүүдийг шүүж шинэ array буцаана

```
const food = [ 🍎, 🥕, 🍌, 🍆, 🌽, 🍓, 🥔, 🥑 ]
```

```
const fruits = food.filter( (item) => item.type === 'fruit' );
```

```
fruits → [ 🍎, 🍌, 🌽, 🥑 ]
```

```
const vegetables = food.filter( (item) => item.type !== 'fruit' );
```

```
vegetables → [ 🥕, 🍆, 🍓, 🥔 ]
```

```
array.filter( (element) => {  
  /* code */  
  return /* a condition */  
});
```

# Array.find()

- Array-ийн элемент бүр дээр эхнээс нь авхуулаад шинэ array-д буулгадаг
- Гэхдээ filter-ээс ялгаатай нь хамгийн түрүүнд олдсон элементийг буцаадаг
- Хэрэв элемент олдоогүй бол Undefined буцаана.

```
const food = [ 🍎, 🥕, 🍌, 🍆, 🍌, 🍎, 🍌, 🥑 ]
```

```
const banana = food.find( (item) => item.color === "yellow" );
```

```
banana → 🍌
```

```
const watermelon = food.find( (item) => item === 🍉 );
```

```
watermelon → undefined
```

---

```
array.find( (element) => {  
  /* code */  
  return /* a condition */  
});
```

# Array.findIndex()

- Array-ийн элемент бүр дээр эхнээс нь авхуулаад шинэ array-д буулгадаг
- Гэхдээ find-аас ялгаатай нь хамгийн түрүүнд олдсон элементийн индексийг буцаадаг.
- Хэрэв элемент олдоогүй бол -1 буцаана.

```
const food = [ 🍎, 🥕, 🍌, 🍆, 🍌, 🍎, 🍌, 🥑 ]

const bananaIndex = food.findIndex( (item) => item.color === "yellow" );

bananaIndex → 4

const watermelonIndex = food.findIndex( (item) => item === 🍉 );

watermelonIndex → -1
```

---

```
array.findIndex((element) => {
    /* code */
    return /* a condition */
});
```

# Array.some()

- Array-д тухайн нөхцлийг хангах элемент ядаж нэг байгаа эсэхийг шалгаж true эсвэл false утга буцаана
- Логик OR үйлдэлтэй ижил.
- Мөн адил нөхцөл буцаана.

```
const food = [🍎, 🥕, 🍌, 🍆, 🌽, 🍅, 🥔, 🥑]
```

```
const hasABanana = food.some( (item) => item === '🌽' );
```

```
hasABanana → true
```

```
const hasAWaterMelon = food.some( (item) => item === '🍉' );
```

```
hasAWaterMelon → false
```

---

```
array.some((element) => {  
  /* code */  
  return /* a condition */  
});
```

# Array.sort()

- Шууд array дээр үйлдэл хийж өөрчилдөг
- Default-оороо array-г string-д шилжүүлж цагаан толгойн дараалалаар эрэмбэлдэг.
- Тийм учраас тоог эрэмблэх callback утга авч эрэмблэх функцийг зааж өгөх хэрэгтэй.

```
const months = ['Jan', 'March', 'April', 'June'];  
months.sort() → [ 'April', 'Jan', 'June', 'March' ]
```

```
const numbers = [1, 5, 80, 9, 6];  
numbers.sort() → [ 1, 5, 6, 80, 9 ]  
                '80', '9'
```

```
numbers.sort((a, b) => { return a - b });    numbers → [ 1, 5, 6, 9, 80 ]
```

`array.sort();`

`array.sort((a, b) => { /* ... */ });`

The **first** element  
for comparison.

The **second** element  
for comparison.

`> 0`    sort a **after** b

`< 0`    sort a **before** b

`=== 0`    **keep** original order of a and b



# Array.reduce()

```
const numbers = [1, 2, 3, 4];
```

```
function reducer (previousValue, currentValue) {  
  return previousValue + currentValue;  
}
```

```
const sum = numbers.reduce(reducer);
```

```
sum → 10
```

| Callback iteration | previousValue | currentValue |
|--------------------|---------------|--------------|
| First call         | 1             | 2            |
| Second call        | 3             | 3            |
| Third call         | 6             | 4            |

`array.reduce( callbackFn , initialValue )`

*reducer*

*optional*

*callbackFn*

```
function name (previousValue, currentValue) {  
  /* code goes here */  
  return /* value */  
}
```

# Array.reduce()

- Array-д ямар нэгэн өөрчлөлт оруулахгүйгээр хоорондох элементүүдийг нэмж буцаадаг.
- Хоёр variable авдаг: Эхнийх нь callbackFunction бол хоёр дахь нь сонголтоор өмнөх утгыг зааж өгч болдог
- Callback Function нь өмнөх утга ба дараагийн утга гэсэн хоёр хувьсагч авч ямар нэгэн үйлдэл хийгээд утга буцаадаг
- Энэ method нь олон утга авч нэг үр дүнг гаргах зорилгоор ашиглагддаг
- Жишээ нь: array-н элементүүдийн нийлбэрийг олох, тоолох, array-г объект болгох гэх мэт олон үйлдэл функцд нь бичиж хэрэгжүүлж болно.
- Ашиглахдаа array.reduce(хэрэглэх функцийн нэр, анхны утгын нэр) –ийг өгч ашиглана. Хэрэв анхны утга оноогоогүй бол default-оор array-н эхний элементийг анхны утга болгон хэрэглэдэг.

# Array.JSON.stringify()

- Хамгийн энгийнээр одоо байгаа json датаг local storage-д хадгалахад string хэлбэрт шилжүүлдэг

- **Object→String**

```
const user = {  
  name: "Tom",  
  age: 25  
};  
  
const jsonString = JSON.stringify(user);  
  
console.log(jsonString);
```

- Үр дүн:

```
{"name": "Tom", "age": 25}
```

# Array.JSON.parse()

- Stringify-ийн эсрэг буюу local storage-оос дата уншиж ашиглахдаа хэрэглэдэг.

- **String→Object**

```
const jsonString = '{"name": "Tom", "age": 25}';  
  
const user = JSON.parse(jsonString);  
  
console.log(user);
```

- Үр дүн:

```
{ name: 'Tom', age: 25 }
```

Let's test our  
knowledge!

# Ecommerce-ийн өгөгдөл боловсруулах

```
const jsonProducts = `[
  {"id":1,
   "name":"Laptop",
   "price":1200,
   "stock":true
  },
  {"id":2,
   "name":"Phone",
   "price":800,
   "stock":true
  },
  {"id":3,
   "name":"Mouse",
   "price":25,
   "stock":false
  },
  {"id":4,
   "name":"Monitor",
   "price":300,
   "stock":true}]`;
```

- 1) Дээрх JSON string-ийг JS массив (array) болгон хувиргах.
- 2) Зөвхөн бэлэн байгаа (**stock: true**) бараануудыг шүүж авах.
- 3) Шүүж авсан бараануудын үнийг 10% хямдруулсан шинэ жагсаалт үүсгэх (Зөвхөн **name** болон шинэ **price**-ийг агуулсан массив байх).
- 4) Хямдарсан бараануудын нийт үнийн нийлбэрийг олох.
- 5) Анхны жагсаалтаас "Phone" гэдэг барааг олж, мөн түүний массивын индексийг тодорхойлох.
- 6) Анхны жагсаалтад 1000-аас дээш үнэтэй бараа байгаа эсэхийг, мөн бүх бараа бэлэн байгаа эсэхийг шалгах.
- 7) Бараануудыг нэрээр нь цагаан толгойн дарааллаар эрэмбэлэх.
- 8) Бараа тус бүрийн нэрийг консол дээр хэвлэж харуулах.
- 9) Хамгийн сүүлд хямдруулсан барааны массивыг буцаагаад JSON string болгон хувиргах.